

UCS503P – Software Engineering Project

Athena

An AI-Powered Campus Query Bot

1024030219 Armandeep Singh Taggar
1024031048 Sanchit Narula
1024031169 Nipun Gupta

Group: 3C73 BE Third Year, CSE

Under the guidance of

Ms. Jhonsy Bansal

Prototype Stage Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

September 17, 2026

Contents

1	Introduction	1
1.1	Background and Context	1
1.2	Project Objectives	1
1.3	Prototype Scope	2
2	Methodology and Design	3
2.1	System Architecture	3
2.2	Software Development Life Cycle	9
3	Results and Evaluation	11
3.1	Testing Methodology	11
3.2	Performance Metrics	11
3.3	Prototype Status and Analysis	13
3.4	Limitations and Threats to Validity	13
4	Conclusion	14
4.1	Summary of Work	14
4.2	Key Findings	14
4.3	Future Work and Recommendations	15
4.4	Final Remarks	15
	Bibliography	16

Chapter 1

Introduction

1.1 Background and Context

Students routinely seek information distributed across institutional websites, academic handbooks, departmental notices, hostel guidelines, examination circulars and placement portals. Finding an authoritative, up-to-date answer can be time-consuming, particularly for students who are unfamiliar with their institution’s information systems.

Physical navigation presents a related challenge. A student may know which laboratory, faculty office or examination hall they need to visit without knowing its location. On a large campus, building names, floor numbers and room labels are not always sufficient to identify a destination intuitively.

Athena brings information access and campus wayfinding into a single web application. It combines institution-specific question answering with a 2D campus map in the initial version (v1), with interactive 3D navigation planned for a subsequent release. Students submit questions in natural language, receive responses based on approved institutional sources, and view mapped destinations for supported location queries.

1.1.1 Problem Statement

A general-purpose language model alone is not an authoritative source for institution-specific details such as fee deadlines, hostel rules or room allocations. Likewise, a static campus map does not directly connect a student’s natural-language request to a building, floor or room.

The project therefore addresses the following problem: *how can a shared campus-assistance platform provide institution-scoped answers and location resolution while restricting access to authorised members of each institution?* Athena’s proposed solution combines Retrieval-Augmented Generation (RAG), affiliation verification and structured campus-location data. Retrieval supplies relevant source material to the generation stage rather than relying on the language model’s parameters alone [LPP+20].

1.2 Project Objectives

1. **Grounded and relevant answers.** Build a retrieval-augmented query bot that answers questions from approved institutional documents. Evaluate factual support against retrieved evidence and collect student feedback on relevance using a curated pilot query set.

2. **Verified, institution-scoped access.** Use Django Auth and institutional-email verification to restrict protected data to authorised users. Begin with a single-college pilot while retaining institution identifiers in records and retrieval filters for later multi-college use.
3. **Campus location resolution.** Resolve building, floor and room queries using structured JSON location data and highlight the result on a 2D SVG or floor-plan map. Treat Three.js navigation and Blender-authored 3D assets as subsequent deliverables, not current v1 functionality.
4. **Incremental, deployable delivery.** Use weekly iterations and automated build and test checks to deliver the core workflow quickly, then release repeatable increments to staging and evaluate user feedback.

The objectives prioritise time-to-value: a working, measurable single-college workflow precedes broader campus coverage and additional features. The assessment criteria in Chapter 3 distinguish implementation scope from demonstrated performance.

1.3 Prototype Scope

The v1 technical baseline comprises a React.js frontend built with Vite and styled with Tailwind CSS, a Django and Django REST Framework (DRF) backend, PostgreSQL with pgvector, Sentence Transformers embeddings and a custom Python RAG pipeline using GPT-4o-mini. PyMuPDF supplies text extraction for institutional PDFs. Django Auth, institutional-email verification and Django Admin support student access and knowledge-base maintenance.

Campus navigation initially uses 2D SVG maps or floor-plan images with structured JSON location mappings. The deployment targets are Vercel for the frontend and Render or Railway for the backend, with a PostgreSQL service supporting pgvector. These are the technologies and deliverables specified for v1; operational readiness and numerical performance remain subject to documented tests.

Subsequent scope includes Three.js and Blender-based 3D navigation, personalised dashboards and reminders, richer multi-college administration, voice interaction, a mobile application, ERP and attendance integration, and personalised academic assistance. No completed implementation or measured benefit is assumed for these roadmap features.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

Athena uses a multi-tier web architecture. A student interacts with the React interface, which sends requests to Django REST Framework endpoints. Django Auth establishes the user's identity; affiliation checks bind protected operations to a verified institution. A client-supplied institution identifier is not, by itself, permission to access that institution's data.

For a knowledge query, the custom Python RAG pipeline embeds the question using Sentence Transformers and retrieves relevant chunks from PostgreSQL through pgvector. The search is restricted to the authorised institution. Retrieved context is passed to GPT-4o-mini to generate the response. A location query instead resolves a destination in institution-scoped JSON records and returns its map identifier and 2D coordinates to the frontend. Requests containing both intents can combine the two results.

2.1.2 Technical Stack

Table 2.1 records the project-specific technology choices. A v1 classification identifies the initial technical baseline, not evidence that a deployment or performance target has already been validated.

2.1.3 UML Design Views

Unified Modeling Language (UML) is used here to describe complementary aspects of the design: the use case diagram identifies user interactions, the class diagram models domain structure, the activity diagram describes control flow, and the sequence diagram specifies message ordering. These diagrams describe the intended design; they are not reverse-engineered evidence of completed code. The SDLC diagram separately documents the development process.

2.1.4 Use Case Diagram

Students authenticate and verify institutional affiliation before submitting questions or viewing campus locations. Academic and event questions use the same grounded-query workflow. College administrators maintain approved knowledge, institutional records and location mappings through

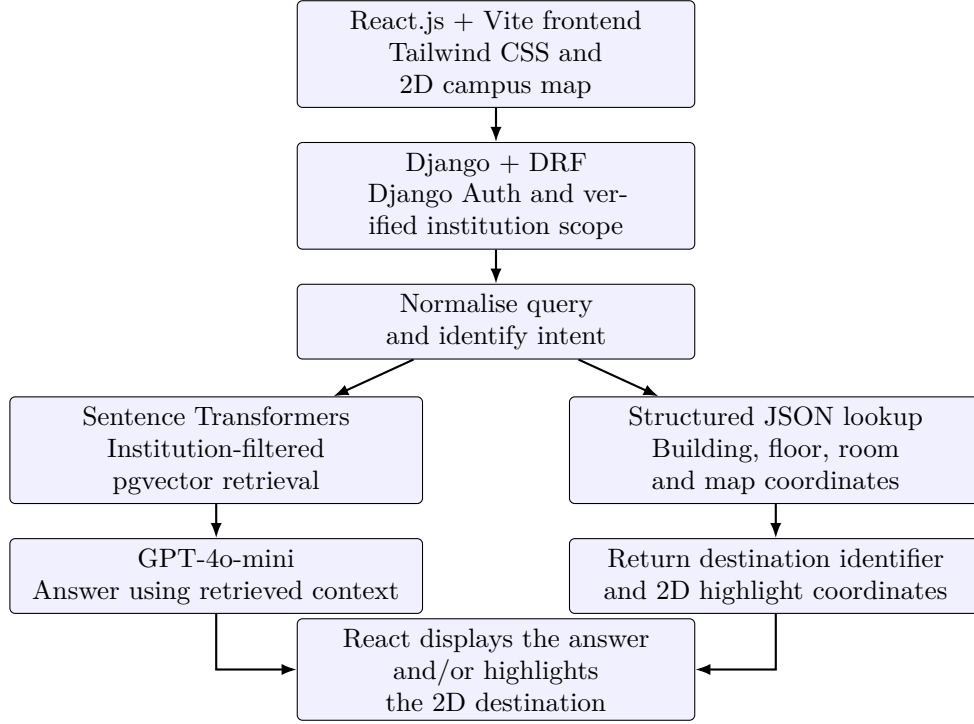


Figure 2.1: v1 request architecture: Django/DRF, institution-scoped RAG and 2D navigation.

Component	Technology	Responsibility and scope
Web interface	React.js, Vite, Tailwind CSS	Student/admin screens and UI styling (v1).
Backend API	Python, Django, DRF	Authentication, domain logic and REST endpoints (v1).
Relational storage	PostgreSQL	Institutions, students, faculty, courses, events, notices and document metadata (v1).
Vector retrieval	pgvector	Store chunk embeddings and perform institution-scoped similarity search (v1).
Embedding model	Sentence Transformers	Embed document chunks and student queries (v1).
Answer generation	GPT-4o-mini	Generate responses from retrieved context through a custom Python RAG pipeline (v1).
Document ingestion	PyMuPDF	Extract text from institutional PDF documents before chunking and embedding (v1).
Access control	Django Auth; institutional email check	Authenticate users, verify affiliation and authorise institution-scoped access (v1).
Administration	Django Admin	Maintain institutional records and approved knowledge documents (v1).
Campus navigation	SVG / floor-plan images; structured JSON	Resolve and highlight buildings, floors and rooms on a 2D map (v1).
Testing and delivery	pytest, Postman, Git, GitHub	Unit/API tests, version control and automated build/test workflow (v1).
Hosting targets	Vercel; Render / Railway	Frontend hosting, backend staging and PostgreSQL with pgvector support (v1 targets).
3D extension	Three.js, Blender	Interactive 3D scenes and campus assets (roadmap).

Table 2.1: Technical baseline and roadmap aligned with the project report.

authorised administrative interfaces. Personal reminders are a later extension and are shown separately as roadmap scope.

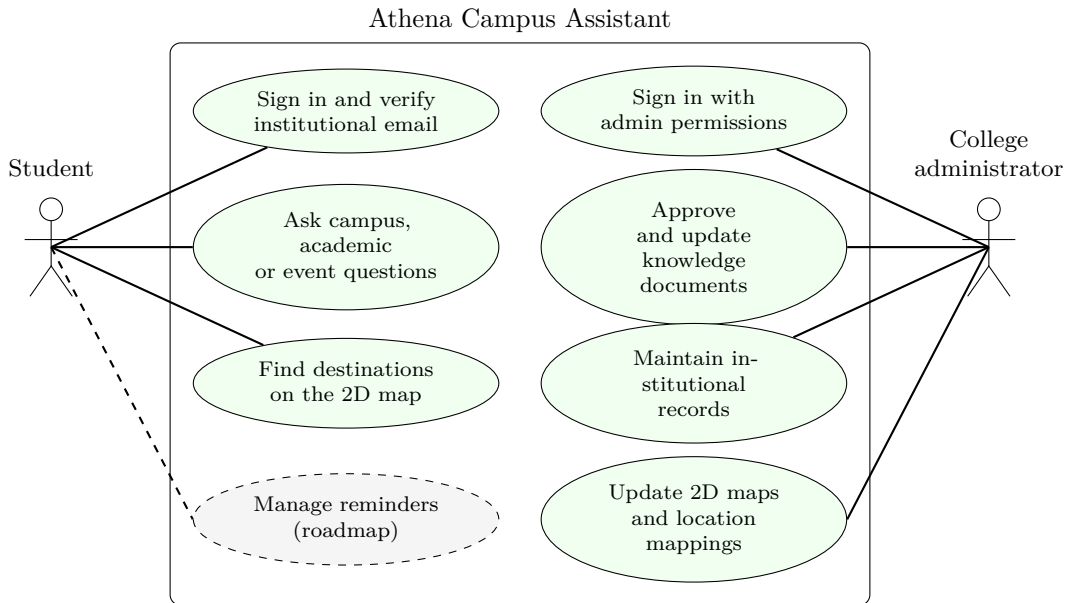


Figure 2.2: UML use case view. Protected actions require verified affiliation or administrative permissions; dashed scope is planned.

2.1.5 Class Diagram

Figure 2.3 models the principal institutional entities together with application-level behaviours. Student and administrator accounts use Django Auth; a profile or membership links an account to its institution. Knowledge documents supply chunks for retrieval, courses are associated with faculty, and societies organise events. The Reminder class is a roadmap extension. Operations shown are design responsibilities, not claims about exact method names in an existing implementation.

Institution identifiers scope faculty, courses, events and societies as well as documents and accounts. Only selected associations are drawn to keep the model legible. Each knowledge document may produce multiple chunks; pgvector stores the corresponding embeddings with links to the document and institution. This is distinct from assigning one embedding to an entire document.

For navigation, the v1 report specifies structured JSON rather than a separate relational building/floor/room schema. The proposed location contract comprises `institutionId`, `locationId`, `building`, `floor`, `room`, `mapId` and 2D coordinates. Campus identifiers must remain consistent across event records, JSON mappings and SVG or floor-plan assets.

2.1.6 Activity Diagram

Figure 2.4 shows the primary flow for a single-intent query. Unauthorised requests terminate before retrieval or location lookup. The two successful processing branches converge only after their respective results have been prepared. Ambiguous locations and insufficient document evidence should produce clarification or an explicit inability to answer, rather than an unsupported destination or response.

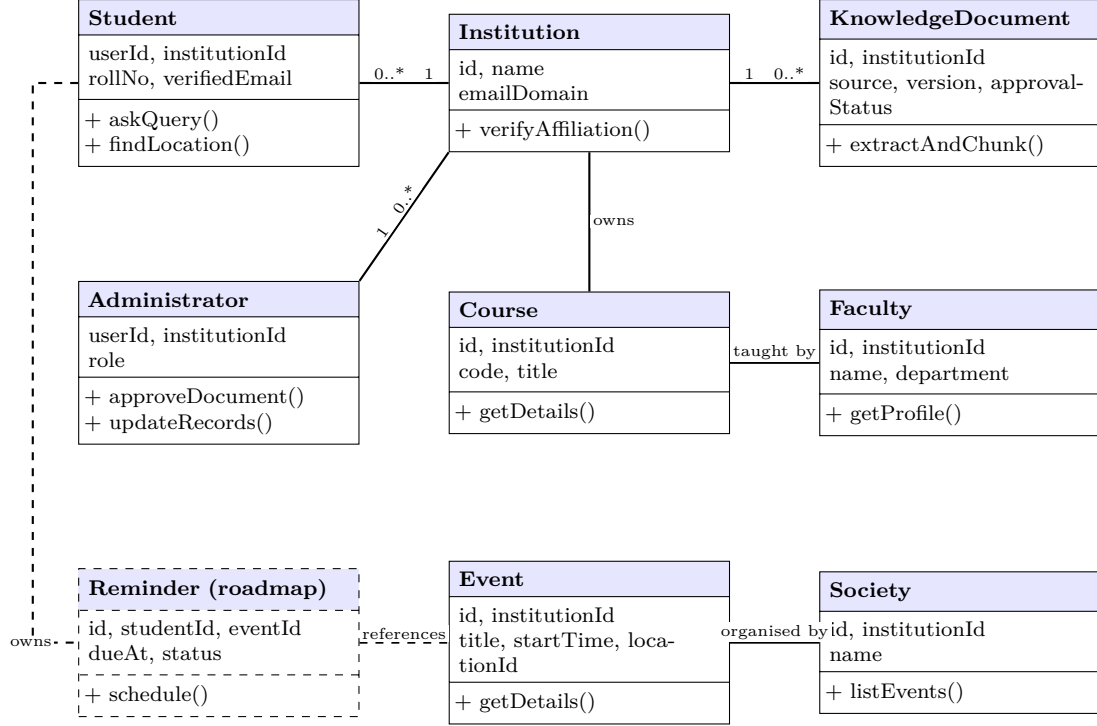


Figure 2.3: UML class model for institutional knowledge and campus services; the dashed reminder extension is outside v1.

2.1.7 UML Sequence Diagram

Figure 2.5 shows the message order for an authorised knowledge query with sufficient retrieved evidence. Authentication and affiliation checks precede retrieval. The location-only path uses JSON lookup instead of an LLM call; access-denied and insufficient-evidence outcomes are handled by the activity flow and the safeguards described below.

2.1.8 Implementation Details

Module 1: document ingestion and knowledge maintenance. An authorised administrator uploads or updates an approved institutional PDF through the administrative workflow. PyMuPDF extracts text, which the custom Python pipeline normalises and splits into retrievable chunks. Sentence Transformers generates chunk embeddings for storage through pgvector. Document identifiers, source metadata and institution identifiers must be retained so retrieved passages remain attributable and correctly scoped. Document updates should replace or retire obsolete chunks rather than leaving contradictory versions active. Chunk size, overlap and the exact embedding model configuration must be recorded for evaluation; the project report does not fix these values.

Module 2: retrieval-grounded query answering. The DRF endpoint invokes the custom Python RAG pipeline after authorisation. The query is embedded using a configuration compatible with stored chunks, and institution-filtered similarity search returns the top- k passages from pgvector. GPT-4o-mini receives the question and retrieved evidence, not the raw question alone. The response should retain source metadata for inspection. If evidence is absent or inadequate, the application should return an explicit insufficient-information response instead of presenting an unsupported answer. Retrieval and prompting do not guarantee correctness; answer grounding

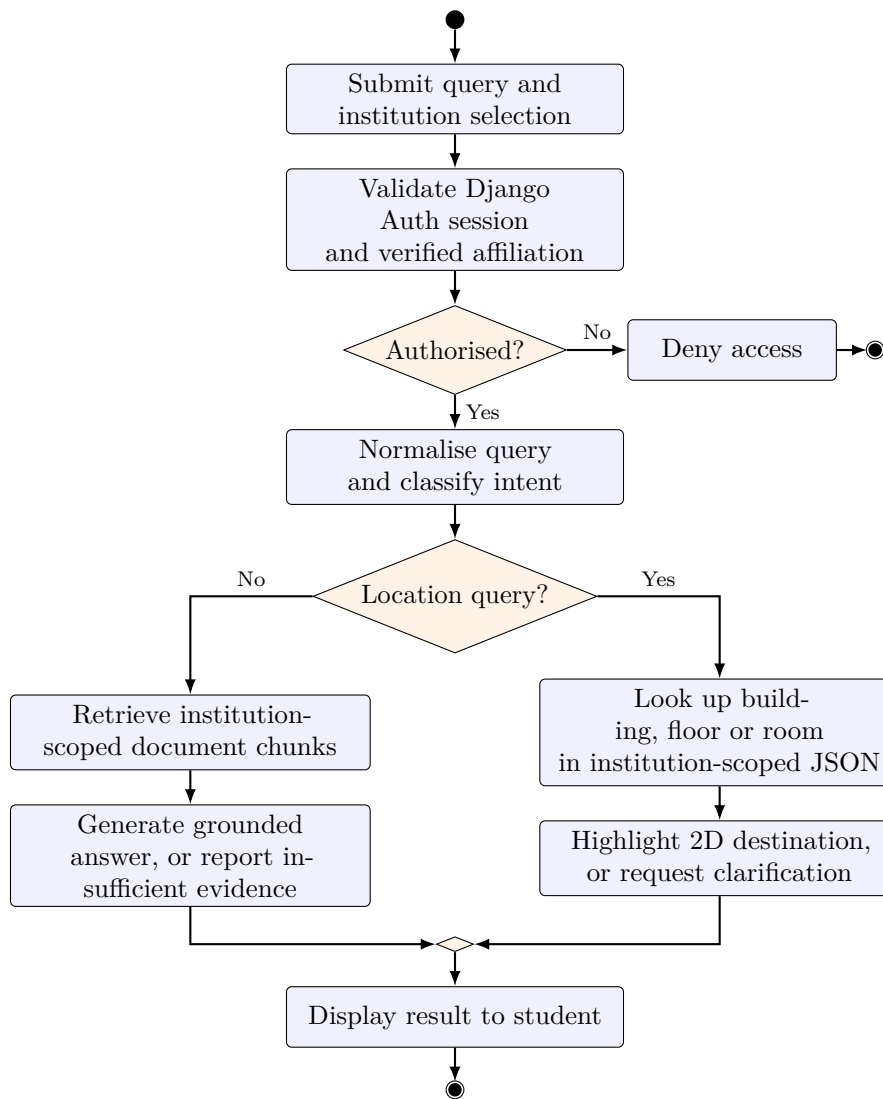


Figure 2.4: Student-query activity flow, including access denial and convergence of the result paths.

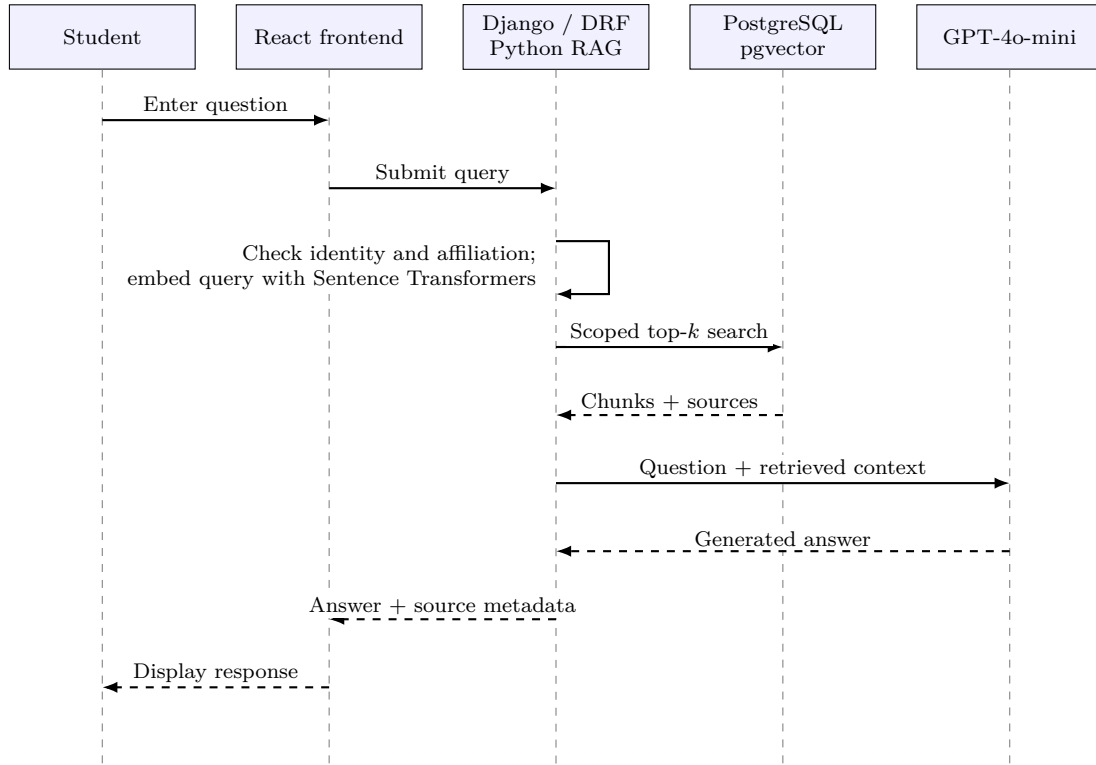


Figure 2.5: UML sequence diagram for the successful v1 RAG query workflow.

and relevance require separate evaluation.

Module 3: authentication and institutional verification. Django Auth supplies the account-authentication layer. The v1 affiliation mechanism is an institutional-email check associated with the user’s authenticated account. The security requirement is to confirm ownership of an approved institutional email address before enabling protected access; a domain string supplied by the client is not sufficient. The report does not specify the email-confirmation implementation, so this remains a validation requirement. Backend authorisation must enforce the verified institution and role on every protected operation. Django Admin permissions restrict document and institutional-record maintenance.

Module 4: 2D campus navigation. The React interface displays SVG campus maps or floor-plan images. Location queries resolve to structured JSON entries describing the building, floor, room, map asset and highlight coordinates. Unknown destinations should return a not-found response; ambiguous names should trigger clarification. Three.js rendering, Blender assets and turn-by-turn directions are roadmap work, not part of the v1 navigation claim.

Module 5: academic information and future personalisation. Institution-scoped course, faculty, event, society and notice records support campus-information queries and administrative maintenance. Personalised student dashboards and date reminders are subsequent deliverables; the class and use case diagrams mark them as planned rather than implemented.

2.1.9 Deployment and CI/CD

The frontend deployment target is Vercel; the Django/DRF backend target is Render or Railway, with PostgreSQL configured to support pgvector. The backend, not the browser, communicates with the external language-model API. Model credentials and database connection settings belong in deployment configuration, not frontend bundles or committed source files.

The planned CI/CD workflow builds the React application and runs backend unit tests on code changes, then produces a repeatable staging release. Postman workflows validate login, affiliation, querying and map lookup after deployment. Git and GitHub provide version control and collaboration; the exact runner configuration and production release policy still need to be documented.

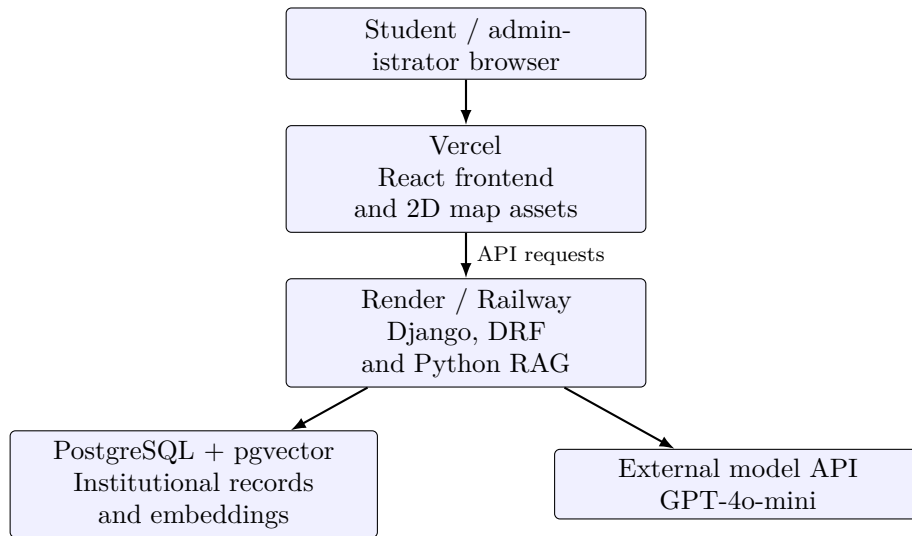


Figure 2.6: Planned deployment responsibilities. Hosting targets do not imply a verified live deployment.

2.2 Software Development Life Cycle

Athena follows an **Agile, iterative** SDLC organised around weekly increments and rapid feedback. The first release prioritises a single-college query workflow over a broad but incomplete feature set. Each iteration refines requirements, updates the design, implements changes, executes automated checks and releases an increment to staging for review.

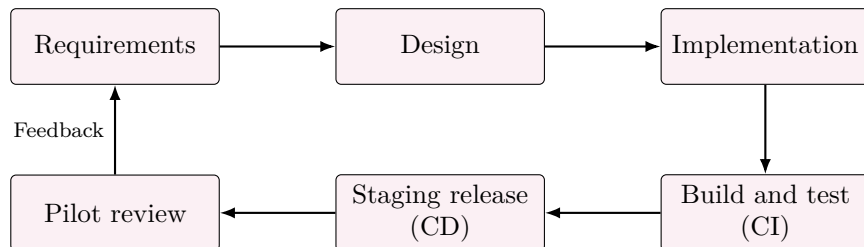


Figure 2.7: Weekly iterative SDLC with continuous integration, staging delivery and pilot feedback.

The proposed increment order is:

1. **Core information workflow:** establish React and Django/DRF, institution records, document ingestion and pgvector-backed answering.
2. **Verified pilot and navigation:** complete institutional-email verification, administrative maintenance and 2D location highlighting; exercise the workflow with one college and collect evaluation evidence.
3. **Expansion after validation:** improve answer quality and staging reliability, then extend institution coverage and introduce roadmap features such as reminders and interactive 3D navigation.

Each increment should have explicit acceptance checks: correct API behaviour, rejection of unauthorised cross-institution requests, source-supported answers, and correct mapped destinations. Sprint numbering and completion dates should be recorded from actual development logs rather than inferred from this plan.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The evaluation separates component correctness, end-to-end behaviour, institution isolation and user-facing quality. The project uses pytest and Postman as its principal test tools; quantitative outcomes remain pending until execution logs, dataset sizes and measured values are documented.

- **Unit testing:** test query normalisation, retrieval-filter construction, affiliation checks and location lookup independently. External model services should be mocked where deterministic assertions are required.
- **Integration testing:** exercise authentication, query submission, context retrieval, answer delivery and location highlighting as connected workflows. Verify behaviour for empty retrieval results, unknown rooms, ambiguous names and service failures.
- **Isolation and authorisation testing:** use at least two test institutions and attempt cross-institution document and location access. Include invalid or expired sessions, unverified email addresses, modified institution identifiers and administrative actions attempted with student privileges.
- **Performance testing:** measure end-to-end query latency under a documented workload. Record model configuration, corpus size, value of k , hardware, network conditions, cache state and concurrency so that measurements can be interpreted and reproduced.
- **Pilot user evaluation:** recruit students and an administrator from one institution. Log query outcomes and collect direct feedback on answer relevance and destination-finding task completion. Compare outcomes across documented iterations; participant counts and results remain pending.

3.2 Performance Metrics

The primary evaluation criterion is **answer groundedness and relevance**: whether an answer is supported by retrieved institutional evidence and useful to the student asking the question. Grounding and student-rated relevance should be reported separately as well as jointly. Secondary metrics cover latency, location resolution, legitimate-student verification and service availability during the declared pilot operating window.

The project report does not supply measured results or fixed numerical acceptance thresholds. These must be established before the pilot and then reported with the test-set size, workload and configuration. Table 3.1 therefore states criteria without presenting proposed targets as achieved values.

Metric	Evaluation criterion	Evidence status
Answer grounding	Factual claims supported by retrieved sources	Source-based review pending
Student-rated relevance	Response addresses the student’s information need	Pilot feedback pending
Joint answer success	Answer is both grounded and judged relevant	Curated-query evaluation pending
Retrieval precision at k	Relevant chunks among the retrieved top- k	Labelled retrieval test pending
Query response time	Submission-to-display latency; report median and 95th percentile	Timing measurements pending
Verification success rate	Legitimate students correctly verified	Positive and negative tests pending
Institution isolation	No successful unauthorised access in the test suite	Adversarial access tests pending
2D navigation accuracy	Correct building, floor or room highlighted	Annotated location tests pending
Service reliability	Login and query availability during declared pilot hours	Staging monitoring pending

Table 3.1: Evaluation criteria for the v1 pilot. All quantitative outcomes await documented measurement.

For a labelled set of N knowledge queries, retrieval precision is defined as

$$P@k = \frac{1}{N} \sum_{i=1}^N \frac{\text{number of relevant chunks among the top } k \text{ for query } i}{k}.$$

Fix k and the relevance criteria before testing. If fewer than k chunks are returned, missing positions count as non-relevant for this metric. Retrieval precision does not measure whether the generated answer is correct or complete.

The *answer grounding rate* is the proportion of substantive answers for which every factual claim is supported by the retrieved institutional evidence. Abstentions are reported separately, together with the proportion of queries receiving a substantive answer, to avoid treating non-answers as successful grounding. Verification success is the proportion of legitimate test users correctly verified. False acceptances among unauthorised users and false rejections among legitimate users should also be reported separately. Perfect performance on a finite suite does not prove security for all possible requests.

Location resolution accuracy is the proportion of annotated, unambiguous, in-scope queries mapped to the correct destination identifier. Unknown or ambiguous destinations should be evaluated separately for appropriate refusal or clarification. Coverage is reported as mapped locations divided by the total locations in a declared campus inventory, with building and room counts stated explicitly.

3.3 Prototype Status and Analysis

The project report specifies the v1 stack and initial deliverables: React and Tailwind screens, Django/DRF endpoints, Django Auth and institutional-email verification, document ingestion, pgvector retrieval, GPT-4o-mini answering, 2D navigation and CI/CD to staging. These define the technical baseline for Objectives 1–4. They are not accompanied by execution logs, deployment checks or quantitative pilot results, so this report does not claim that the acceptance criteria have already been met.

The central integration challenge is to connect ingestion, verified access and retrieval without losing institution scope or source attribution. A separate navigation challenge is to keep location identifiers and 2D map coordinates consistent with campus records. Source refresh, reliable query handling and correct highlighting should be validated before expanding to multiple colleges or introducing 3D assets.

3.4 Limitations and Threats to Validity

The principal limitation is the absence of reported quantitative results and reproducible test artefacts. A small or unrepresentative query set could overestimate answer quality; outdated or incomplete documents could yield answers that are grounded in their sources but no longer correct for students. Knowledge-base approval therefore needs document maintenance and version tracking.

The exact Sentence Transformers model, chunking parameters, retrieval settings, email-verification workflow and deployment configuration remain to be documented. External-model variation and network conditions may affect answer quality and latency. A single-college pilot and limited map coverage cannot establish performance or usability across all institutions. Final evaluation should state the number of documents, queries, mapped locations, institutions and participants, and specify how long staging availability was monitored.

Chapter 4

Conclusion

4.1 Summary of Work

Athena is designed as a web-based campus companion combining verified institutional knowledge with conversational access and location search. Its v1 baseline uses React.js, Vite and Tailwind CSS; Django and DRF; PostgreSQL with pgvector; Sentence Transformers; PyMuPDF; and GPT-4o-mini. Django Auth, institutional-email verification and Django Admin support access and maintenance. Navigation initially uses 2D maps and structured JSON, while Three.js and Blender remain part of the roadmap.

The engineering contribution is the integration of mature components into a measurable, deployable workflow, rather than a claim of novel retrieval algorithms. The use case, class, activity and sequence diagrams describe the system design; the deployment and SDLC views connect that design to incremental delivery. Actual performance remains to be established through a documented pilot.

4.2 Key Findings

- **Institution scope must be enforced throughout the backend.** Authentication, affiliation checks, document retrieval, JSON location lookup and administrative operations must share the same authorised institution context; prompt instructions are not an access-control mechanism.
- **Grounding and usefulness are different requirements.** Source support must be assessed alongside the student’s judgement of relevance, with insufficient-evidence cases recorded explicitly.
- **A 2D-first release limits initial integration scope.** SVG or floor-plan maps and structured location records provide a testable navigation workflow before investment in 3D assets and routing.
- **Fast feedback needs repeatable delivery.** Automated builds, backend tests and staging releases support short iterations whose changes can be evaluated against a consistent pilot workflow.

These are design conclusions and evaluation priorities, not quantitative experimental findings.

4.3 Future Work and Recommendations

1. **Validate the first-college pilot.** Define acceptance thresholds, assemble labelled queries, test email verification and institution isolation, and report groundedness, relevance, latency, navigation accuracy and uptime.
2. **Extend navigation after v1.** Expand the 2D location inventory, then add Three.js scenes and Blender-authored models. Turn-by-turn directions require a connected routing representation in addition to destination coordinates.
3. **Improve knowledge and multi-college administration.** Strengthen approval, versioning and refresh workflows, and develop the planned admin dashboard for maintaining multiple independently scoped institutions.
4. **Add personalised services.** Introduce student dashboards and academic or campus-date reminders after the core query workflow is evaluated.
5. **Broaden interaction and integration.** Explore voice assistance, mobile access, ERP and attendance integration, and personalised academic help.
6. **Harden delivery.** Document Vercel and Render/Railway staging, database migrations, secret management, monitoring and rollback procedures before claiming production reliability or scalability.

4.4 Final Remarks

Athena addresses fragmented campus information through verified access, retrieval-grounded answers and a searchable campus map. A single-college, 2D-first pilot provides a focused starting point, while institution-scoped records and modular web components allow later expansion. The next priority is evidence: documented tests and student feedback should guide each release before broader deployment and roadmap features are introduced.

Bibliography

- [LPP+20] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020, pp. 9459–9474 (cit. on p. 1).